Sisteme de operare
Laborator 5
Comunicare inter-procese, sisteme message passing

1. Creati un nou subdirector *lab5/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu PATH pentru a include directorul *SO/laborator/lab5/bin*.

2. Scrieti un program C **pipe.c** care creaza un *pipe* intre un proces parinte si un copil. Parintele scrie un mesaj in *pipe* si asteapta sa se termine procesul copil. Copilul citeste mesajul trimis de parinte in *pipe* si il afiseaza pe ecran. Primul lucru pe care il fac atat parintele cat si copilul dupa crearea *pipe*-ului este sa inchida partea de *pipe* pe care nu o folosesc la comunicarea dintre ei.

3. Scrieti un program C **sigpipe.c** care modifica programul de la punctul 2 a.i. :
- procesul copil nu inchide capatul sau de scriere in *pipe*
- dupa ce a citit din *pipe* mesajul scris de procesul parinte, copilul scrie inapoi in *pipe* mesajul “done\n” si se termina
- dupa ce a scris in *pipe* primul mesaj pentru copil, parintele asteapta sa citeasca din pipe mesajul “done\n” scris de copil si apoi il afiseaza pe ecran.

Ce se intampla? Cum explicati situatia?

4. Scrieti un program C **vfork-pipe.c** care creeaza un proces copil cu ajutorul apelului sistem *vfork* si creeaza un *pipe* intre cele doua procese. Procesul copil citeste un caracter din *pipe* si il afiseaza pe ecran impreuna cu PID-ul sau. Apoi incheie executia cu succes (cod zero). Procesul parinte scrie caracterul ‘b’ in *pipe* dupa care incheie executia.

Ce se intampla? Cum explicati situatia?

5. Scrieti un program C **my-mkfifo.c** care foloseste apelul *mkfifo* pentru a crea un *named pipe* (fisier de tip *FIFO*) cu numele dat ca argument in linie de comanda. Programul are efectul urmatoarelor comenzi shell:

```
$ mknod <nume fifo> p
```

sau

```
$ mkfifo <nume fifo>
```

Folositi comanda *ls -l* pentru a identifica daca fisierul creat are atribute de fisier *FIFO*.

ex:
```
$ mknod myfifo p
$ ls -l myfifo
```

Apoi, scrieti doua programe C **fifo-echod.c** si **fifo-echo.c** care scriu si citesc dintr-un fisier creat cu una dintre comenzile anterioare (fie cea rezultata prin compilarea programului **my-mkfifo.c**, fie prin comenzile shell de mai sus). **fifo-echod.c** deschide pentru citire si scriere fisierul *FIFO* primit ca parametru in linie de comanda si citeste in bucla caractere din fisierul *FIFO*. Dupa fiecare string de caractere citit din *FIFO*, **fifo-echod.c** trimite inapoi in *FIFO* string-ul citit (i.e., are functionalitate de serviciu de echo). Programul **fifo-echod.c** se lanseaza in background ca in exemplul de mai jos si asteapta sa primeasca mesaje in fisierul *FIFO*:

```
$ gcc -o echod fifo-echod.c
$ echod myfifo &
$ jobs
```

Programul **fifo-echo.c** deschide pentru citire si scriere fisierul *FIFO* primit ca parametru in linie de comanda si citeste in bucla caractere de la tastatura, de ex cu functia *fgets*. Dupa fiecare citire, programul scrie in *FIFO* caracterele citite de la tastatura, asteapta ecoul de la *echod* (i.e. citeste din *FIFO* ceea ce scrie *echod*) si tipareste caracterele pe ecran. Programul ruleaza la infinit pana cand e oprit cu Ctrl-C.

```
$ gcc -o echo fifo-echo.c
$ echo myfifo
```

6. Scrieti un program C **ls-more.c** care emuleaza urmatoarea comanda shell:

```
$ ls -l | less
```

Programul creeaza in procesul parinte un *pipe* si doua procese copil, primul responsabil de executia comenzii *ls -l*, iar cel de-al doilea pentru comanda *less*. Dupa ce a creat procesele copil, procesul parinte se blocheaza asteptand terminarea lor si recupereaza starea lor de terminare.

Indicatie: comanda *ls -l* tipareste output-ul pe ecran, in vreme ce *less* in comanda de mai sus citeste datele de intrare de la *stdin* (nu are ca parametru un nume de fisier). In aceasta situatie e nevoie sa redirectati din program *stdout*-ul lui *ls -l* catre capatul de scriere al *pipe*-ului, respectiv *stdin*-ul lui *less* la capatul de citire al *pipe*-ului. In acest sens, folositi *dup/dup2*.

Nu uitati sa inchideti in fiecare proces capetele de *pipe* care nu sunt folosite.

7. Scrieti doua programe C **msg-echod.c** si **msg-echo.c** care implementeaza tot un serviciu de *echo* ca si la punctul 5, dar de data aceasta folosind cozi de mesaje System V IPC. Practic, in loc sa foloseasca un fisier *FIFO*, programul **msg-echod.c** creeaza o coada de mesaje cu ajutorul *msgget* (parametrul *key* il generati cu functia *ftok*). Apoi apeleaza in bucla functia *msgrcv* pentru a primi mesaje de la clienti pe care le scrie inapoi in coada de mesaje (serviciu de *echo*) folosind *msgsnd*. Programul **msg-echod.c** se lanseaza in background ca serviciu de echo dupa cum urmeaza:

```
$ gcc -o echod  msg-echod.c
$ echod keyfile &                  # keyfile e un fisier pe care il creati voi
$ ipcs -q                          # vizualizati coada creata
```

Programul **msg-echo.c** foloseste parametrul primit in linie de comanda (*keyfile*) ca argument pt functia *ftok* si astfel poate identifica si obtine cu ajutorul *msgget* un ID pentru coada de mesaje creata de *echod*. Apoi, citeste in bucla caractere de la tastatura folosind functia *fgets* si asambleaza mesaje cu tipul dat de PID-ul sau (obtinut cu *getpid*) pe care le trimite in coada de mesaje cu ajutorul functiei *msgsnd*. Apoi citeste din coada de mesaje cu ajutorul *msgrcv* ecoul caracterelor trimise catre serviciul *echod* pe care, odata primite, le tipareste pe ecran. Programul se termina cand utilizatorul apasa Ctrl-C.

```
$ gcc -o echo msg-echo.c
$ echo keyfile
```

La final, nu uitati sa faceti curatenie:

```
$ jobs
$ kill %<n>          # n e numarul de job al echod afisat de comanda
                     # jobs de mai sus

$ ipcs -q
$ ipcrm -q <id>      # id e id-ul cozii de mesaje tiparit de comanda
                     # ipcs de mai sus
```